27 Oktober 2026

Struktur Data

Circular & Double-Ended Queue

Andi Nurkholis, S.Kom., M.Kom.

Jurusan Informatika

Fakultas Teknik Industri

Universitas Pembangunan Nasional “Veteran” Yogyakarta

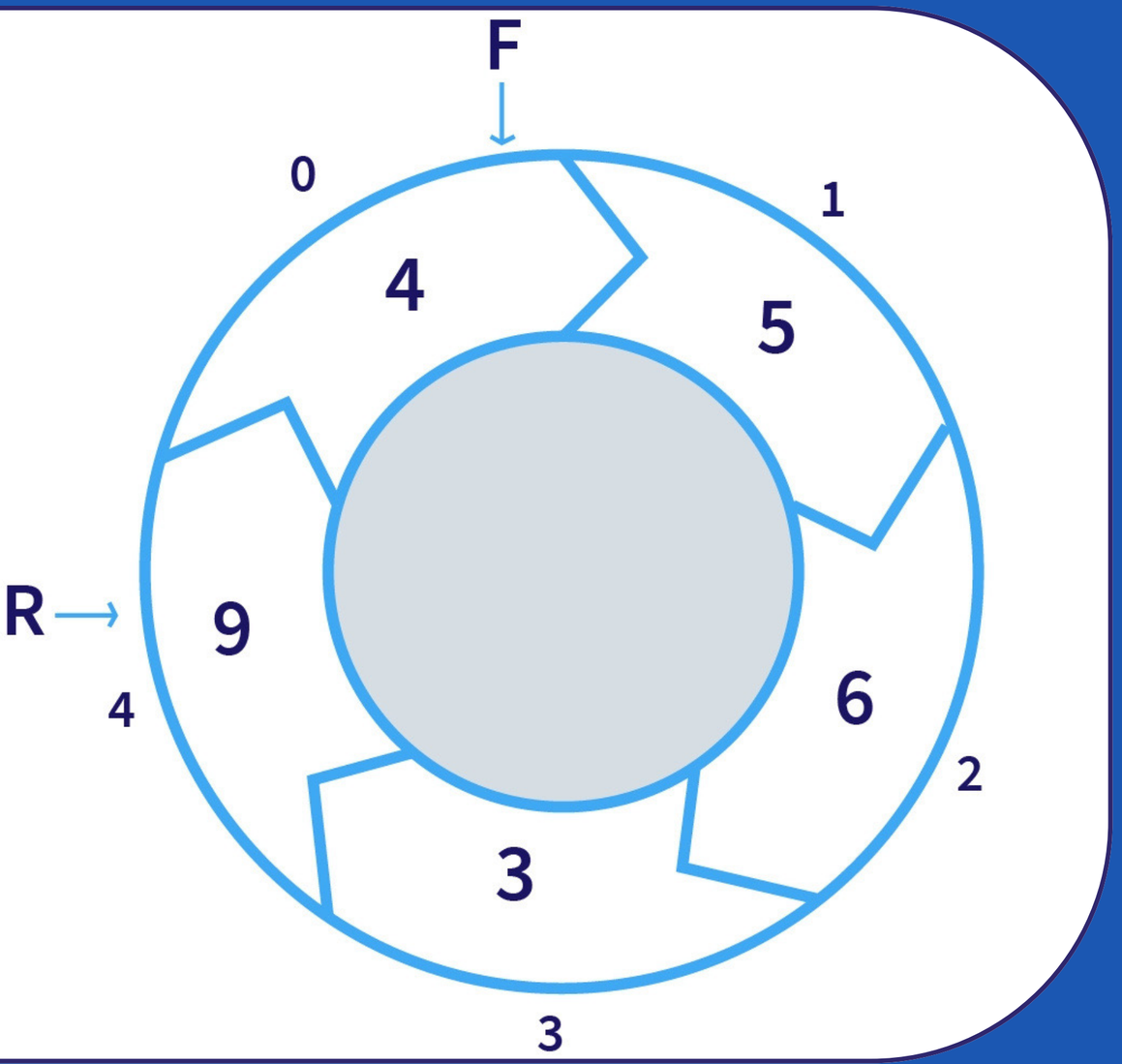


Definisi Circular Queue

Circular Queue (Antrian Melingkar) adalah variasi **queue** yang lebih **efisien** dari pada **linear queue**. **Rear** dapat kembali ke **indeks awal array** ketika mencapai **batas akhir**, selama masih ada **ruang kosong**. Hal ini mencegah **pemborosan memori** akibat **dequeue**. Circular queue tetap menerapkan prinsip **FIFO (First In, First Out)**, namun dengan pemanfaatan **kapasitas** lebih **optimal**. Analogi: kursi melingkar yang bisa ditempati kembali.

Struktur dasar Circular Queue:

```
struct CircularQueue {
    int front, rear;      // indeks depan & belakang
    int size, capacity;  // jumlah elemen & kapasitas maksimum
    int* array;          // penyimpanan elemen
};
```



Karakteristik
Circular Queue

1. Struktur Melingkar
2. FIFO (First In, First Out)
3. Efisiensi Memori
4. Indeks Front dan Rear Dinamis
5. Kondisi Overflow dan Underflow

Karakteristik Circular Queue

1. **Struktur Melingkar**: Rear (ekor) akan kembali ke indeks 0 setelah mencapai indeks terakhir array, selama posisi tersebut belum ditempati. Hal ini menciptakan efek “melingkar” yang memungkinkan pemanfaatan ruang secara optimal.

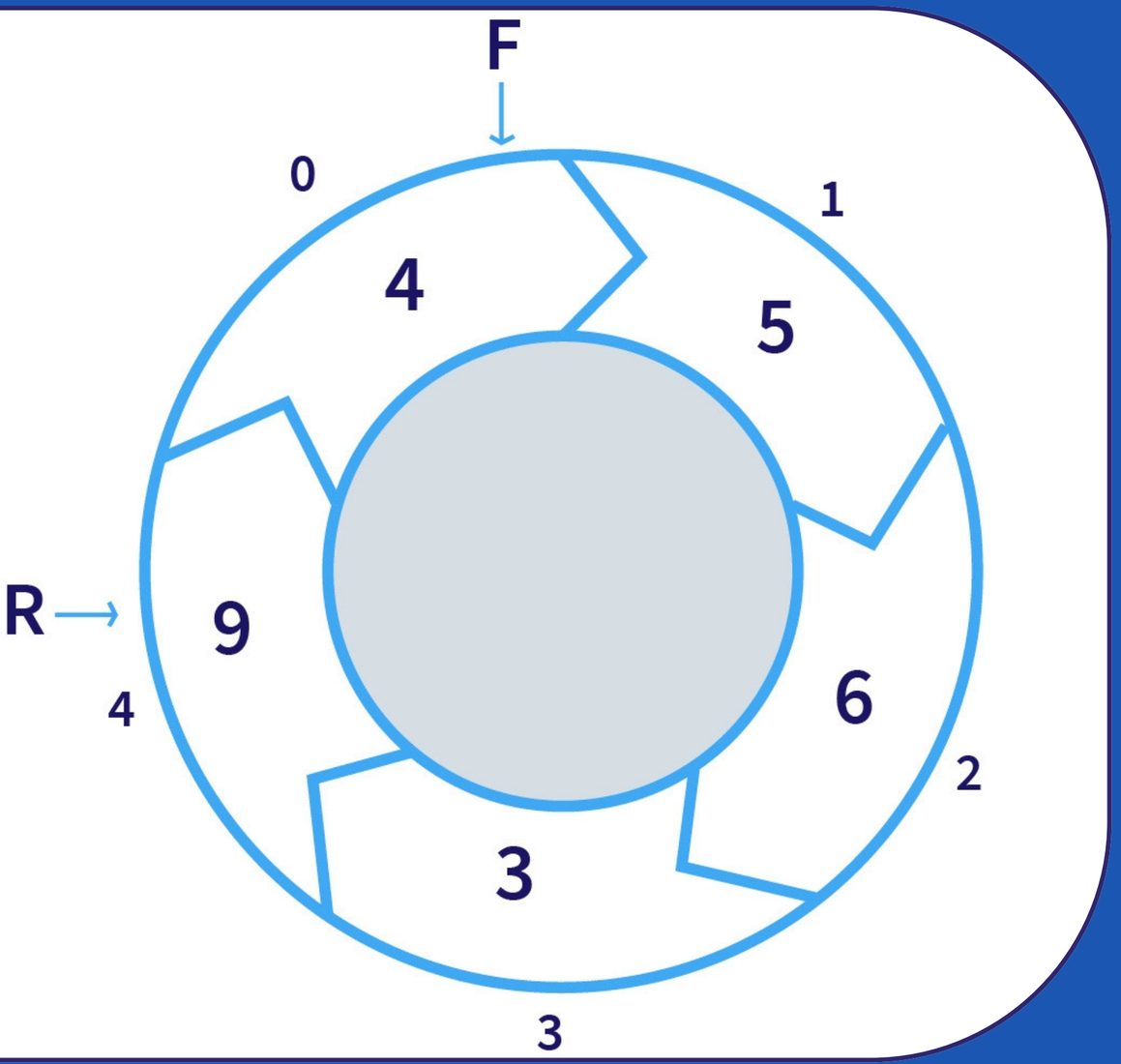
2. **FIFO (First In, First Out)**: Sama seperti queue biasa, circular queue tetap mengikuti prinsip FIFO, di mana elemen yang pertama kali masuk akan menjadi elemen yang pertama kali keluar.

3. **Efisiensi Memori**: Tidak ada ruang array yang terbuang percuma karena slot kosong yang muncul akibat operasi dequeue bisa dimanfaatkan kembali. Hal ini berbeda dengan linear queue yang membuat slot kosong di depan tidak bisa diisi kembali jika rear sudah mencapai ujung.

Karakteristik Circular Queue

4. **Indeks Front dan Rear Dinamis**: Posisi front dan rear akan bergerak melingkar mengikuti operasi enqueue (penambahan) dan dequeue (penghapusan). Ketika rear mencapai batas akhir array, ia akan kembali ke indeks awal jika masih ada ruang kosong.

5. **Kondisi Overflow dan Underflow**: Overflow terjadi jika semua slot sudah terisi meskipun secara indeks rear masih bisa berputar ke awal. Underflow terjadi jika tidak ada elemen di dalam circular queue (kosong).



Operasi Dasar
Circular Queue

1. Enqueue
2. Dequeue
3. Front
4. Rear

Operasi Enqueue

1. **Periksa kondisi penuh**: Jika (rear + 1) % size == front (array), maka queue penuh dan operasi enqueue tidak bisa dilakukan. Pada linked list, kondisi penuh biasanya hanya terjadi jika memori habis.

2. **Tangani kondisi kosong**: Jika front == -1 (array), set front = rear = 0. Jika front == NULL (linked list), buat node baru dan set front = rear = node.

3. **Pindahkan pointer rear**: Jika tidak kosong, pada array perbarui rear = (rear + 1) % size. Pada linked list, buat node baru, tautkan dengan rear->next = node, lalu set rear = node.

4. **Simpan elemen baru**: Masukkan nilai pada queue[rear] (array) atau simpan ke rear->data (linked list).

Operasi Dequeue

1. **Periksa kondisi kosong**: Jika front == -1 (array) atau front == NULL (linked list), maka queue kosong dan operasi dequeue tidak bisa dilakukan.

2. **Ambil elemen terdepan**: Simpan nilai pada queue[front] (array) atau front->data (linked list) sebagai elemen yang dihapus.

3. **Tangani kondisi satu elemen**: Jika front == rear (array) atau front == rear (linked list), set front = rear = -1 (array) atau front = rear = NULL (linked list) agar queue kembali kosong.

4. **Geser posisi front**: Jika masih ada elemen, perbarui front = (front + 1) % size (array) atau front = front->next (linked list) agar menunjuk ke elemen berikutnya.

Operasi Front

1. **Periksa kondisi kosong**: Jika front == -1 (array) atau front == NULL (linked list), maka queue kosong sehingga tidak ada elemen yang bisa diakses.

2. **Identifikasi posisi terdepan**: Pada array, posisi terdepan ditunjuk oleh indeks front, sedangkan pada linked list ditunjuk langsung oleh pointer front.

3. **Akses elemen terdepan**: Ambil nilai pada queue[front] (array) atau front->data (linked list).

4. **Kembalikan nilai**: Hasil dari operasi ini adalah data yang berada pada elemen paling depan queue.

Operasi Rear

1. **Periksa kondisi kosong**: Jika rear == -1 (array) atau front == NULL (linked list), maka queue kosong dan tidak ada elemen yang bisa diakses.

2. **Identifikasi posisi terakhir**: Pada array, posisi terakhir ditunjuk oleh indeks rear, sedangkan pada linked list ditunjuk langsung oleh pointer rear.

3. **Akses elemen terakhir**: Ambil nilai pada queue[rear] (array) atau rear->data (linked list).

4. **Kembalikan nilai**: Hasil dari operasi ini adalah data yang berada pada elemen paling akhir (rear) queue.

Definisi Double-Ended Queue

Double-Ended Queue (Deque) adalah variasi **struktur data antrian** yang memungkinkan operasi **penyisipan (enqueue)** dan **penghapusan (dequeue)** dilakukan dari kedua **ujung**, baik **front** maupun **rear**. Tidak seperti queue biasa yang terbatas pada satu arah, **deque** lebih **fleksibel** karena bisa berfungsi sebagai **stack** dan **queue** sekaligus. Struktur ini bermanfaat dalam buffer management, penjadwalan proses, serta algoritma sliding window.

Struktur dasar Double-Ended Queue:

```
struct Deque {
    int front, rear;      // indeks depan & belakang
    int size, capacity;  // jumlah elemen & kapasitas maksimum
    int* array;          // penyimpanan elemen
};
```



Karakteristik
Double-Ended Queue

1. Akses dari Kedua Ujung
2. Dua Variasi Utama Deque
3. Kapasitas dan Ukuran Dinamis
4. Kemiripan dengan Stack dan Queue
5. Kompleksitas Operasi

Karakteristik Double-Ended Queue

1. **Akses dari Kedua Ujung**: Elemen dapat ditambahkan (insert) maupun dihapus (delete) baik dari ujung depan (front) maupun ujung belakang (rear). Hal ini menjadikan deque lebih fleksibel dibanding queue konvensional.

2. **Dua Variasi Utama Deque**: 1) Input-Restricted Deque: Penyisipan hanya boleh dilakukan dari satu ujung (misalnya rear), tetapi penghapusan dapat dilakukan dari kedua ujung (front dan rear). 2) Output-Restricted Deque: Penghapusan hanya boleh dilakukan dari satu ujung (misalnya front), tetapi penyisipan dapat dilakukan dari kedua ujung (front dan rear).

Karakteristik Double-Ended Queue

3. **Kapasitas dan Ukuran Dinamis**: Deque dapat diimplementasikan dengan array atau linked list. Array terbatas dan rentan overflow/underflow, sedangkan linked list lebih fleksibel karena panjangnya dapat bertambah sesuai kebutuhan.

4. **Kemiripan dengan Stack dan Queue**: Deque bisa berperilaku sebagai stack jika hanya menggunakan satu ujung (misalnya front untuk push dan pop). Deque juga bisa berperilaku sebagai queue jika hanya membatasi operasi penyisipan di rear dan penghapusan di front.

5. **Kompleksitas Operasi**: Semua operasi pada deque seperti insertFront, insertRear, deleteFront, dan deleteRear umumnya dapat dilakukan dalam waktu konstan O(1), baik menggunakan array atau linked list.



Operasi Dasar
Double-Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear

Operasi Enqueue Front

1. **Periksa kondisi penuh**: Jika (front == 0 && rear == size - 1) || (front == rear + 1) (array), maka deque penuh dan operasi enqueue front tidak bisa dilakukan. Pada linked list, kondisi penuh hanya terjadi jika memori habis.

2. **Tangani kondisi kosong**: Jika kosong, set front = rear = 0 (array) atau buat node baru lalu front = rear = node (linked list).

3. **Geser posisi front**: Jika tidak kosong, pada array perbarui front = (front - 1 + size) % size agar tetap melingkar ke belakang. Pada linked list, buat node baru, tautkan node->next = front, lalu set front = node.

4. **Simpan elemen baru**: Masukkan nilai ke queue[front] (array) atau simpan ke front->data (linked list).

Operasi Enqueue Rear

1. **Periksa kondisi penuh**: Jika (front == 0 && rear == size - 1) || (front == rear + 1) (array), maka deque penuh dan enqueue rear tidak bisa dilakukan. Pada linked list, penuh hanya jika memori habis.

2. **Tangani kondisi kosong**: Jika kosong, set front = rear = 0 (array) atau buat node baru lalu front = rear = node (linked list).

3. **Geser posisi rear**: Jika tidak kosong, pada array perbarui rear = (rear + 1) % size. Pada linked list, buat node baru, tautkan rear->next = node, lalu set rear = node.

4. **Simpan elemen baru**: Masukkan nilai ke queue[rear] (array) atau simpan ke rear->data (linked list).

Operasi Dequeue Front

1. **Periksa kondisi kosong**: Jika front == -1 (array) atau front == NULL (linked list), maka deque kosong dan operasi dequeue front tidak bisa dilakukan.

2. **Ambil elemen terdepan**: Simpan nilai pada queue[front] (array) atau front->data (linked list) sebagai elemen yang dihapus.

3. **Tangani kondisi satu elemen**: Jika front == rear (array) atau front == rear (linked list), set front = rear = -1 (array) atau front = rear = NULL (linked list) agar deque kembali kosong.

4. **Geser posisi front**: Jika masih ada elemen, perbarui front = (front + 1) % size (array) atau front = front->next (linked list) agar menunjuk ke elemen berikutnya.

Operasi Dequeue Rear

1. **Periksa kondisi kosong**: Jika rear == -1 (array) atau rear == NULL (linked list), maka deque kosong dan operasi dequeue rear tidak bisa dilakukan.

2. **Ambil elemen terakhir**: Simpan nilai pada queue[rear] (array) atau rear->data (linked list) sebagai elemen yang dihapus.

3. **Tangani kondisi satu elemen**: Jika front == rear (array) atau front == rear (linked list), set front = rear = -1 (array) atau front = rear = NULL (linked list) agar deque kembali kosong.

4. **Geser posisi rear**: Jika masih ada elemen, perbarui rear = (rear - 1 + size) % size (array). Pada linked list, geser rear ke node sebelumnya dengan melintasi dari front hingga node sebelum rear.

27 Oktober 2026

Sekian

Terima Kasih



Andi Nurkholis, S.Kom., M.Kom.

Jurusan Informatika

Fakultas Teknik Industri

Universitas Pembangunan Nasional “Veteran” Yogyakarta